STUDENTSPHERE

A Cloud-Based Student Management and Synchronization Platform

CLOUD COMPUTING LABORATORY PROJECT REPORT

(22IT580)

Submitted by

SHANMUGA PERUMAL A - Register Number: 2403917720521313

VANAMAMALAI M - Register Number: 2403917720521086

EDWIN MOSES G - Register Number: 2403917720521305

Submitted in partial fulfilment of the requirements for the Cloud Computing Laboratory

Department of Information Technology

Thiagarajar College of Engineering, Madurai

Academic Year: 2026-2027

CERTIFICATE

This is to certify that the project report entitled **"StudentSphere: A Cloud-Based Student Management and Synchronization Platform"** is the work carried out by **Shanmuga Perumal A (2403917720521313), Vanamamalai M (2403917720521086), and Edwin Moses G (2403917720521305)** in partial fulfilment of the requirements for the Cloud Computing Laboratory (22IT580), Department of Information Technology, Thiagarajar College of Engineering, Madurai, during the academic year 2026-2027.

PROJECT GUIDE	COURSE FACULTY	HEAD OF THE DEPARTMENT
Signature: _____	Signature: _____	Signature: _____
Name: _____	Name: _____	Name: _____

Department of Information Technology
Thiagarajar College of Engineering, Madurai

DECLARATION

We hereby declare that the project report entitled **"StudentSphere: A Cloud-Based Student Management and Synchronization Platform"** is prepared by us for the Cloud Computing Laboratory. The report describes the application present in the accompanying project folder and its implemented features. Sources and platform documentation consulted during the work are listed in the References section.

STUDENT	REGISTER NUMBER	SIGNATURE
Shanmuga Perumal A	2403917720521313	_____
Vanamamalai M	2403917720521086	_____
Edwin Moses G	2403917720521305	_____

Place: Madurai
Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of Thiagarajar College of Engineering and the Department of Information Technology for providing the academic environment and facilities required to undertake this project.

We thank our project guide and Cloud Computing Laboratory faculty for their guidance, feedback, and encouragement during the design and development of StudentSphere. We also acknowledge the documentation and developer tools provided by Firebase, Auth0, Dropbox, and MongoDB, which helped us study cloud authentication, storage, and database integration.

Shanmuga Perumal A
Vanamamalai M
Edwin Moses G

ABSTRACT

StudentSphere is a browser-based student record management application developed to demonstrate how a familiar administrative workflow can be connected to cloud identity, storage, and database services. The interface provides account sign-in and registration, a dashboard of student records, search and department filtering, student details, and add/delete operations. Each student record contains an identifier, name, age, and department.

The application is implemented with HTML, CSS, and JavaScript modules. Firebase Authentication is used for email/password and Google sign-in; Auth0 is available as an alternate identity-provider integration. Student data can be cached in the browser and synchronized with Firebase Firestore and Cloud Storage. Optional integrations provide JSON backup to Dropbox and document synchronization to MongoDB Atlas. For hosted deployments, a Firebase Cloud Function verifies Firebase ID tokens and reads the MongoDB connection URI from Secret Manager before accessing Atlas.

The project demonstrates modular frontend development, cloud service integration, persistence and fallback strategies, authenticated API communication, and user-oriented CRUD workflows. Integrations depend on the corresponding provider accounts, configuration, and security rules being enabled; the application can also use a local demonstration mode where supported.

Keywords: Student management, Firebase, Firestore, Cloud Storage, Auth0, Dropbox, MongoDB Atlas, cloud computing.

TABLE OF CONTENTS

LIST OF TABLES . . . 8
1. INTRODUCTION . . . 9
1.1 Background . . . 9
1.2 Problem Statement . . . 9
1.3 Objectives . . . 9
1.4 Scope . . . 9
2. RELATED WORK AND CLOUD CONTEXT . . . 9
2.1 Traditional Record Management . . . 9
2.2 Cloud Service Concepts . . . 9
2.3 Motivation for Multiple Backends . . . 9
3. FEASIBILITY STUDY . . . 10
3.1 Technical Feasibility . . . 10
3.2 Operational Feasibility . . . 10
3.3 Economic Feasibility . . . 10
4. REQUIREMENTS ANALYSIS . . . 10
4.1 Hardware and Software Requirements . . . 10
4.2 Functional Requirements . . . 10
4.3 Non-Functional Requirements . . . 11
5. EXISTING AND PROPOSED SYSTEM . . . 11
5.1 Existing Approach . . . 11
5.2 Proposed StudentSphere System . . . 11
6. TECHNOLOGIES AND TOOLS . . . 12
6.1 Development Environment . . . 12
7. SYSTEM ARCHITECTURE . . . 12
7.1 Architecture Layers . . . 12
7.2 Request and Synchronization Flow . . . 13
8. SYSTEM DESIGN AND MODULES . . . 13
8.1 Authentication Module . . . 13
8.2 Student Management Module . . . 13
8.3 Cloud Provider Modules . . . 13
8.4 Settings and Notifications . . . 13
9. DATA DESIGN . . . 13
9.1 Example JSON Record . . . 14
9.2 Persistence and Fallback . . . 14
10. IMPLEMENTATION . . . 14
10.1 User Interface and Dashboard . . . 14
10.2 Authentication and Account Flow . . . 15

10.3 Student Record Operations . . . 16
10.4 Search and Department Filter . . . 17
10.5 Firebase Data Integration . . . 18
10.6 Dropbox Integration . . . 18
10.7 MongoDB Atlas Integration . . . 18
11. USER INTERFACE . . . 19
11.1 Login and Registration . . . 19
11.2 Dashboard . . . 19
11.3 Dialogs and Feedback . . . 19
12. TESTING AND VALIDATION . . . 19
12.1 Acceptance Test Scenarios . . . 19
12.2 Validation Considerations . . . 20
13. DEPLOYMENT . . . 20
13.1 Local Development . . . 20
13.2 Firebase Hosting and Cloud Function . . . 20
13.3 Third-Party Provider Configuration . . . 20
14. SECURITY AND PRIVACY . . . 20
14.1 Authentication and API Access . . . 20
14.2 Credential Handling . . . 21
14.3 Data Considerations . . . 21
15. LIMITATIONS . . . 21
15.1 Current Limitations . . . 21
16. FUTURE ENHANCEMENTS . . . 21
16.1 Planned Improvements . . . 21
17. CONCLUSION . . . 21
18. REFERENCES . . . 22

LIST OF FIGURES

FIGURE	DESCRIPTION
1	StudentSphere login and account registration page
2	Student dashboard and student records table
3	Add student form
4	Student details view
5	Delete student confirmation
6	Search and department filter
7	Dropbox storage configuration panel
8	MongoDB Atlas settings and synchronization panel
9	High-level StudentSphere architecture

LIST OF TABLES

TABLE	DESCRIPTION
1	Student record data fields
2	Functional requirements
3	Cloud service responsibility mapping
4	Representative acceptance test cases

1. INTRODUCTION

1.1 Background

Student information is commonly maintained in spreadsheets or separate departmental systems. Such approaches can make it difficult to find records quickly, keep backups, and provide consistent access to authorized users. StudentSphere demonstrates a compact web-based alternative that organizes basic student records in one searchable interface and connects the application to cloud identity and persistence services.

1.2 Problem Statement

The project addresses the need for a simple interface to create, inspect, search, and remove student records while illustrating how data can be cached locally and backed by configurable cloud services. It also demonstrates separating user authentication from storage providers so the frontend can support multiple service integrations.

1.3 Objectives

- Build a responsive browser interface for student record administration.
- Provide sign-in, registration, and sign-out flows with cloud identity options.
- Support student creation, details display, search, department filtering, and deletion.
- Show dashboard metrics for total students, average age, and unique departments.
- Demonstrate local caching and optional synchronization with cloud storage and databases.
- Protect the hosted MongoDB API with Firebase token verification and server-side secrets.

1.4 Scope

The current record schema is intentionally small: student ID, name, age, and department. The project focuses on a single application dashboard and configurable persistence integrations. It is a laboratory demonstration, not a full institutional student information system; role-based administration, enrollment, grades, attendance, and audit workflows are outside the current scope.

2. RELATED WORK AND CLOUD CONTEXT

2.1 Traditional Record Management

Manual registers and spreadsheets are easy to start with but offer limited concurrent access, backup automation, and integrated authentication. A web application can centralize the common operations and can delegate identity and persistence to managed cloud services.

2.2 Cloud Service Concepts

StudentSphere combines managed application services with a browser interface. Firebase Authentication and Auth0 provide identity services; Firestore and MongoDB Atlas provide managed database options; Firebase Cloud Storage and Dropbox provide file-oriented storage and backup. Firebase Hosting and Cloud Functions support deployment of the frontend and a protected API endpoint.

2.3 Motivation for Multiple Backends

The integrations provide a practical comparison of document databases and file-based JSON storage. A browser cache can keep the dashboard responsive, while cloud providers can persist records beyond a single browser. Optional provider integrations are configured independently so the core user interface does not depend on every provider being available at once.

3. FEASIBILITY STUDY

3.1 Technical Feasibility

The project uses standard browser technologies and JavaScript modules, a small Node.js HTTP server for local development, and cloud SDKs or REST APIs for provider integrations. The project folder includes the frontend, server, Firebase Hosting configuration, and Cloud Function source. Service use requires valid provider configuration and network access.

3.2 Operational Feasibility

The dashboard uses familiar account forms, summary cards, filters, and a student table. Dialogs support record creation, detail viewing, and deletion confirmation. Toast notifications communicate operation status and errors.

3.3 Economic Feasibility

The software stack is based on commonly available developer tools and managed cloud offerings. Actual operating costs depend on provider plans, storage, function invocations, and usage. MongoDB Cloud Functions deployment requires a Firebase billing-enabled project, so deployment cost and quotas should be reviewed before production use.

4. REQUIREMENTS ANALYSIS

4.1 Hardware and Software Requirements

- A computer capable of running a modern web browser and Node.js.
- A current browser such as Chrome, Edge, or Firefox with JavaScript enabled.
- Node.js and npm for the local HTTP server and dependency management.
- Firebase project configuration for Firebase Authentication, Firestore, Storage, Hosting, and Functions as required.
- Optional Auth0 application, Dropbox developer app, and MongoDB Atlas cluster for the corresponding integrations.

4.2 Functional Requirements

The primary user-facing capabilities and cloud integration responsibilities are summarized below.

ID	REQUIREMENT
FR-01	Register and sign in using configured email/password identity.
FR-02	Support Google sign-in through the enabled Firebase or Auth0 flow.
FR-03	Display student records and aggregate dashboard metrics.
FR-04	Search records by student name, ID, or department.
FR-05	Filter records by department and inspect a selected record.
FR-06	Add a student with ID, name, age, and department.
FR-07	Ask for confirmation before deleting a student record.
FR-08	Cache records locally and synchronize to configured cloud providers.
FR-09	Allow provider settings and connection checks through the settings UI.

4.3 Non-Functional Requirements

- Usability: provide clear labels, confirmation dialogs, empty states, and feedback.
- Responsiveness: update the local view quickly while cloud synchronization runs.
- Availability: tolerate an unavailable optional provider by using other configured sources or local cache.
- Security: keep the MongoDB URI server-side and verify Firebase ID tokens in the hosted function.
- Maintainability: separate authentication, data access, provider configuration, and UI behavior into modules.

5. EXISTING AND PROPOSED SYSTEM

5.1 Existing Approach

A basic manual or spreadsheet workflow generally requires users to locate records by opening files, maintain separate backup copies, and manage access outside the data interface. The approach may be adequate for a small set of records but provides little integrated search, identity handling, or cloud sync.

5.2 Proposed StudentSphere System

StudentSphere presents a single dashboard for common record operations. Authentication is handled through configured identity services, and the data layer can use local cache together with Firebase, Dropbox, or MongoDB Atlas. The application exposes settings to configure optional services and test connections, while the hosted MongoDB integration uses a token-protected Cloud Function.

EXISTING APPROACH	PROPOSED APPROACH
Records handled in separate files or spreadsheets.	Records presented in one searchable student dashboard.
Backups are typically copied manually.	Configured cloud providers can synchronize or back up student data.
Access management is disconnected from the records.	Sign-in is integrated with Firebase Authentication and optional Auth0.
Finding a department or student requires manual filtering.	Client-side search and department filtering update the table.

6. TECHNOLOGIES AND TOOLS

TECHNOLOGY	ROLE IN THE PROJECT
HTML5	Structure for the authentication, dashboard, forms, dialogs, and settings.
CSS3	Responsive layout, typography, forms, cards, tables, and dialogs.
JavaScript ES modules	Application behavior, authentication, data management, and provider bridges.
Node.js HTTP server	Serves the local frontend and exposes development MongoDB API routes.
Firebase Authentication	Email/password and Google identity flows.
Firebase Firestore / Cloud Storage	Cloud persistence and JSON backup for student data.
Auth0	Optional hosted identity provider and Google connection.
Dropbox Files API v2	Optional upload/download of the students.json backup.
MongoDB Atlas	Optional cloud document database for student records.
Firebase Cloud Functions	Hosted MongoDB API with Firebase ID-token verification.

6.1 Development Environment

The application source is organized as browser modules and can be served locally with the provided Node.js server. The frontend loads Firebase web SDK modules from the official Firebase CDN. The Cloud Function is configured for the Node.js 22 runtime in firebase.json.

7. SYSTEM ARCHITECTURE

StudentSphere uses a browser-first architecture. The JavaScript interface calls authentication and data modules, updates the local cache, and invokes configured cloud integrations. During local development, server.js serves the static application and exposes MongoDB routes. In Firebase Hosting, requests under /api/mongodb are rewritten to the mongodbApi Cloud Function.

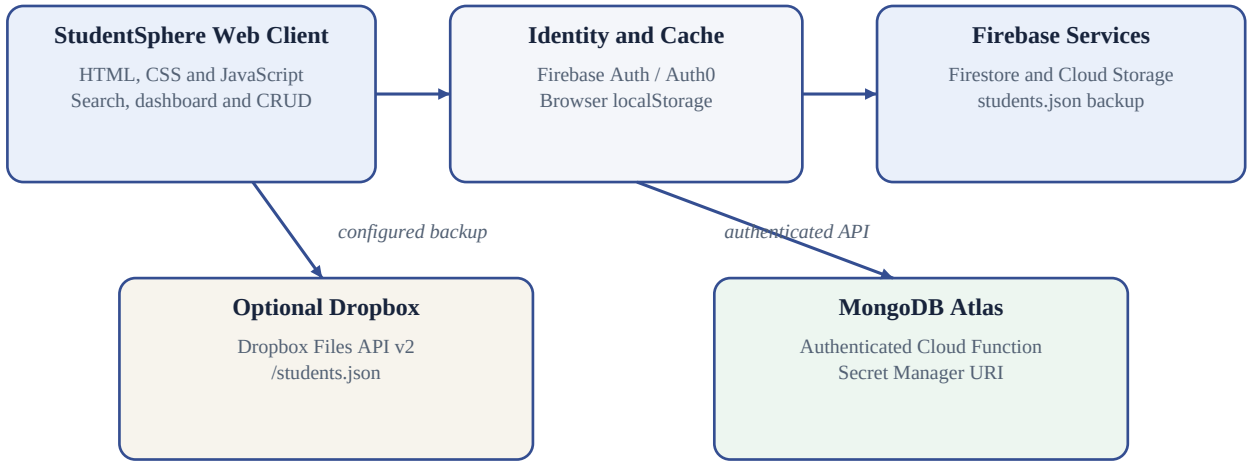


Figure 9: High-level StudentSphere architecture and optional cloud integrations.

7.1 Architecture Layers

- **Presentation layer:** HTML, CSS, and app.js render the sign-in experience, dashboard, table, and modal dialogs.

• **Application services:** authentication modules, student-data.js, and provider service modules implement workflows.

• **Local persistence:** browser localStorage retains a student cache and selected runtime configuration.

• **Cloud persistence:** Firestore and Storage are used through the Firebase web SDK; Dropbox and MongoDB are optional integrations.

• **Protected server integration:** the hosted function verifies Firebase identity and reads MONGODB_URI from Secret Manager.

7.2 Request and Synchronization Flow

On dashboard load, the data module attempts to obtain records from the configured cloud sources and falls back to the browser cache or a small initial demonstration dataset when necessary. When a record is added, local cache is updated first; provider synchronization is initiated separately so the interaction need not wait for each remote service. Delete operations similarly update the local dataset and invoke configured provider deletion paths.

8. SYSTEM DESIGN AND MODULES

8.1 Authentication Module

auth-service.js initializes Firebase Authentication when valid configuration and the Firebase SDK are available. It supports email/password sign-in and registration, Google sign-in, sign-out, and an offline/local demonstration mode. auth0-service.js provides the alternate Auth0 Google sign-in flow and redirect-session handling.

8.2 Student Management Module

student-data.js defines the student record flow, reads and writes the local cache, and coordinates provider reads and synchronization. app.js handles form submission, table rendering, student detail dialogs, deletion confirmation, summary metrics, search, and department filtering.

8.3 Cloud Provider Modules

Provider configuration is separated from its service implementation. Firebase configuration, Auth0 settings, Dropbox settings, and MongoDB database/collection settings are represented in distinct modules. The Dropbox service calls the Dropbox API, while mongodb-service.js calls the application API and the server/function code performs database operations.

8.4 Settings and Notifications

The settings dialog contains provider panels for Firebase, Auth0, Dropbox, and MongoDB. Users can save or reset settings, test selected connections, and request MongoDB synchronization. Toast notifications provide concise success, information, and error feedback.

9. DATA DESIGN

A student record is normalized around four user-entered fields. MongoDB writes may also attach an updatedAt timestamp. The database collection is named students in the studentsphere_db database by default.

FIELD	TYPE	DESCRIPTION	EXAMPLE
id	String	Student identifier; used as a stable record key.	STU-101

FIELD	TYPE	DESCRIPTION	EXAMPLE
name	String	Student's display name.	Alex Morgan
age	Integer	Student age in years.	21
department	String	Academic department.	Computer Science
updatedAt	ISO timestamp	Synchronization update time in MongoDB.	Generated by service

9.1 Example JSON Record

```
{
  "id": "STU-101",
  "name": "Alex Morgan",
  "age": 21,
  "department": "Computer Science"
}
```

9.2 Persistence and Fallback

The browser cache is stored under the students_firebase_cache localStorage key. The Firebase file backup uses students.json. Dropbox defaults to /students.json. MongoDB stores one document per student and uses the id field for upsert and delete matching. The data module attempts cloud sources and uses cached or initial records when no populated cloud result is available.

10. IMPLEMENTATION

10.1 User Interface and Dashboard

index.html defines the application views and controls, while app.js attaches event handlers and renders the live dashboard. The KPI cards show record count, average age, and the number of unique departments. Student rows provide detail and delete actions, and the table displays a result count for the current filter.

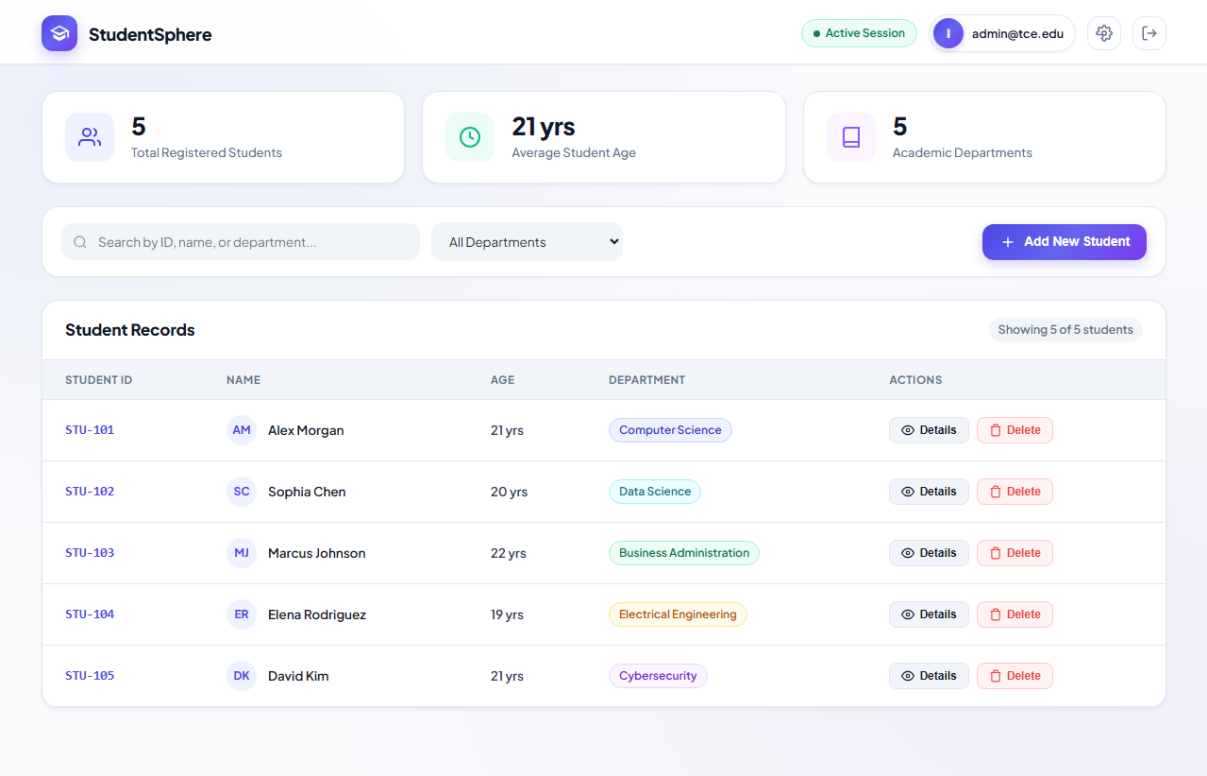


Figure 2: StudentSphere dashboard and student records.

10.2 Authentication and Account Flow

The authentication screen includes sign-in and registration tabs, required email and password fields, and a Google sign-in option. Registration captures a display name and checks matching password inputs in the UI workflow. When Firebase is unavailable, the authentication service can fall back to a local demonstration account store.

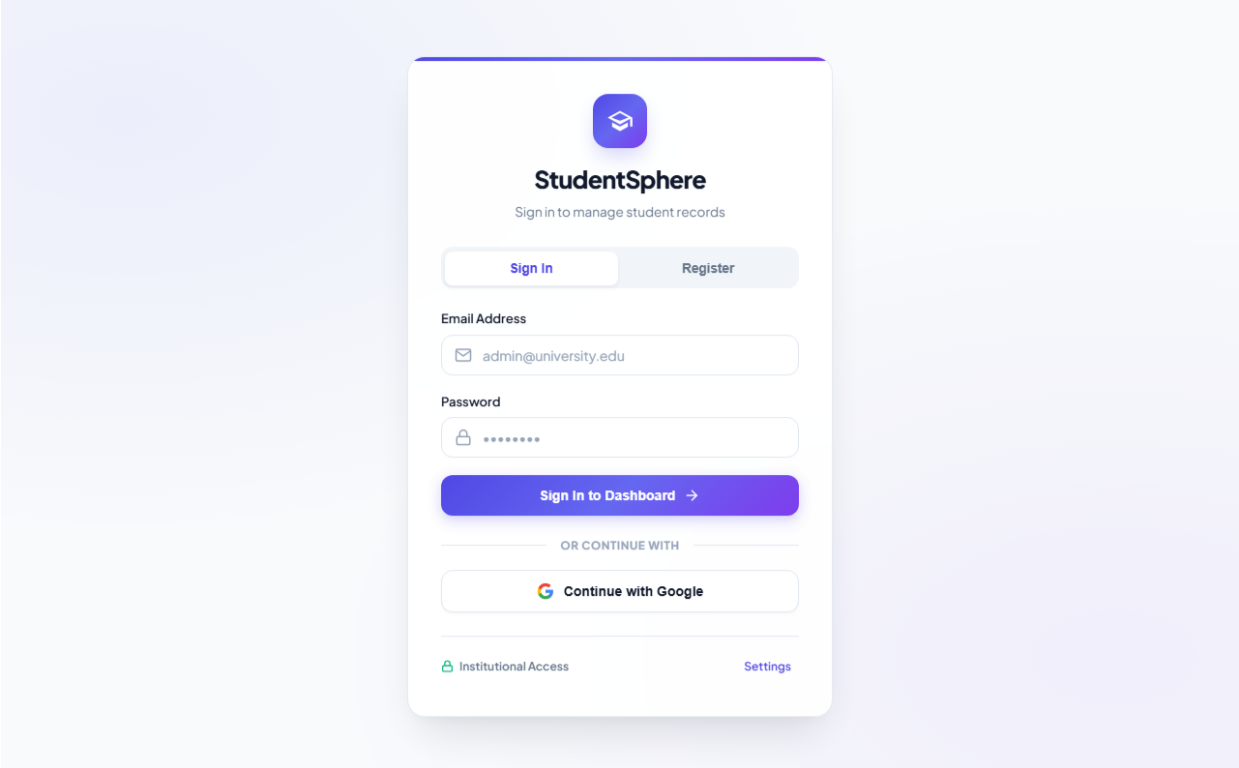


Figure 1: StudentSphere login and registration interface.

10.3 Student Record Operations

The add form collects an optional student ID, name, age, and department. If an ID is omitted, the data module creates a short STU-prefixed ID. Details are displayed in a modal; deletion requires explicit confirmation before the data module removes the selected record.

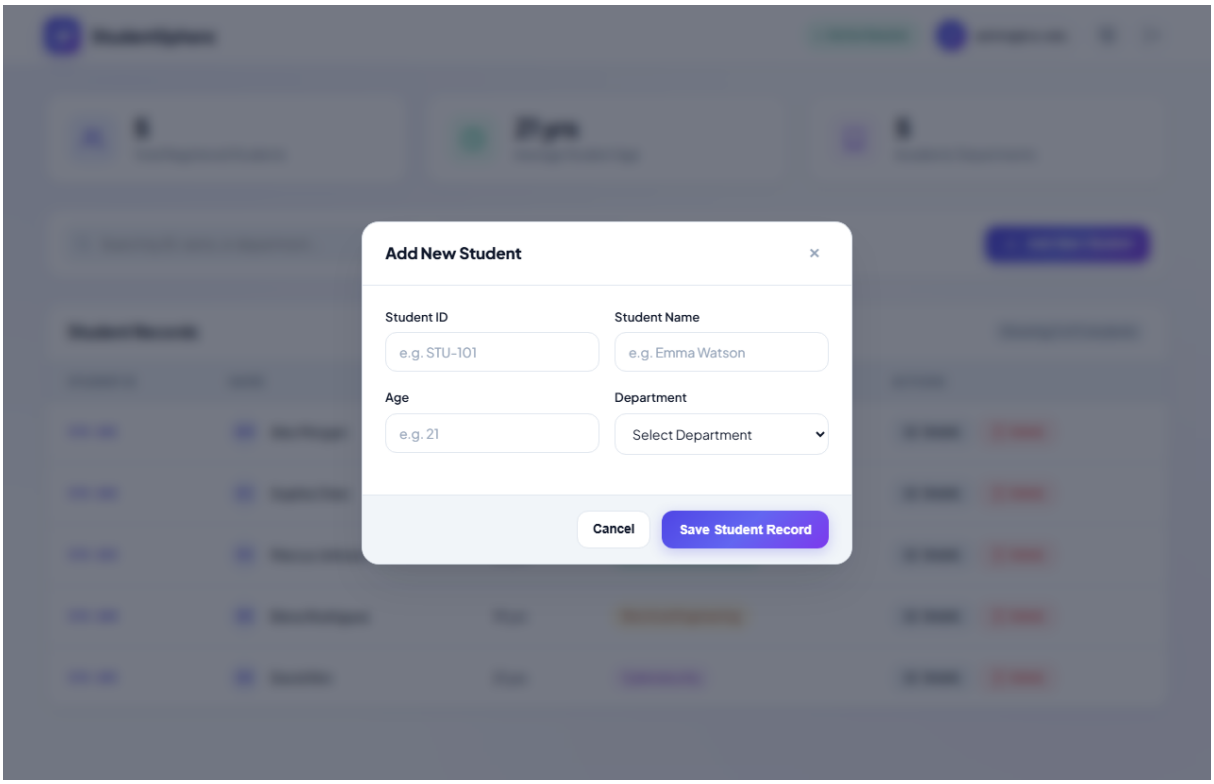


Figure 3: Add student form.

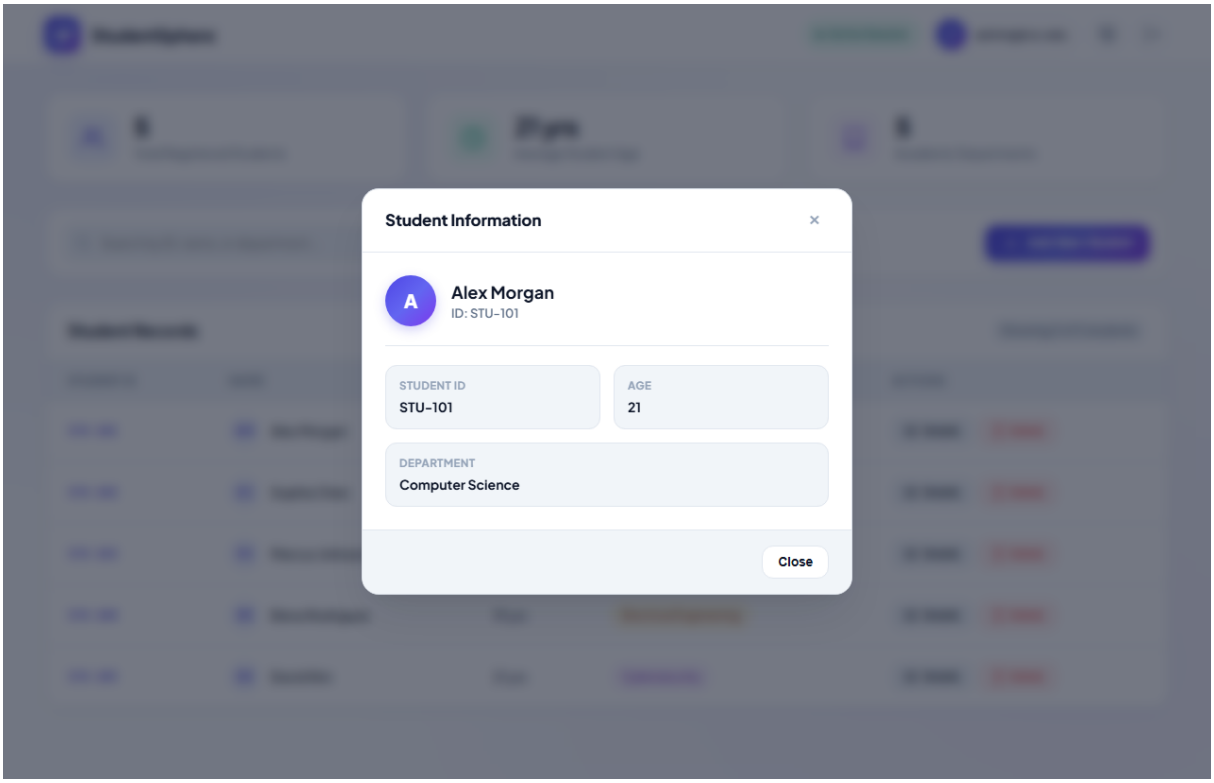


Figure 4: Student details view.

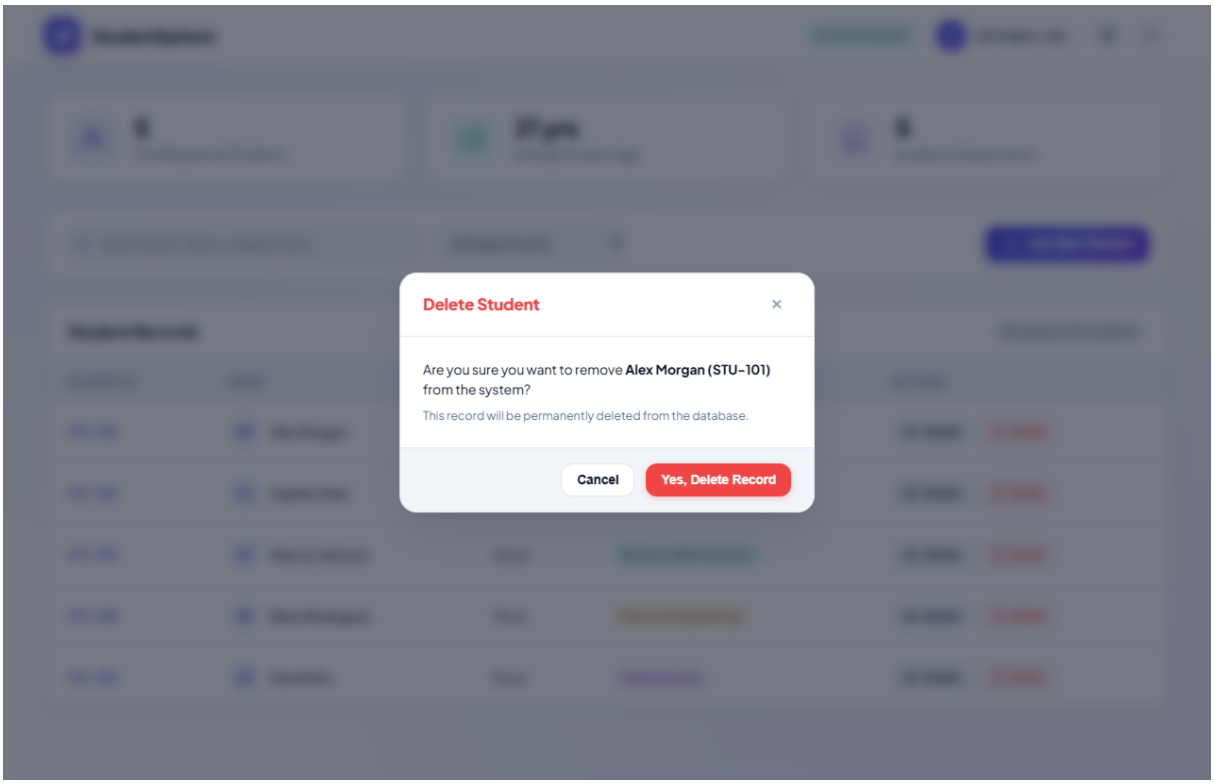


Figure 5: Delete confirmation dialog.

10.4 Search and Department Filter

Search matches case-insensitively against student name, ID, and department. The department selector can be combined with text search; the table and result counter update as the user changes either input.

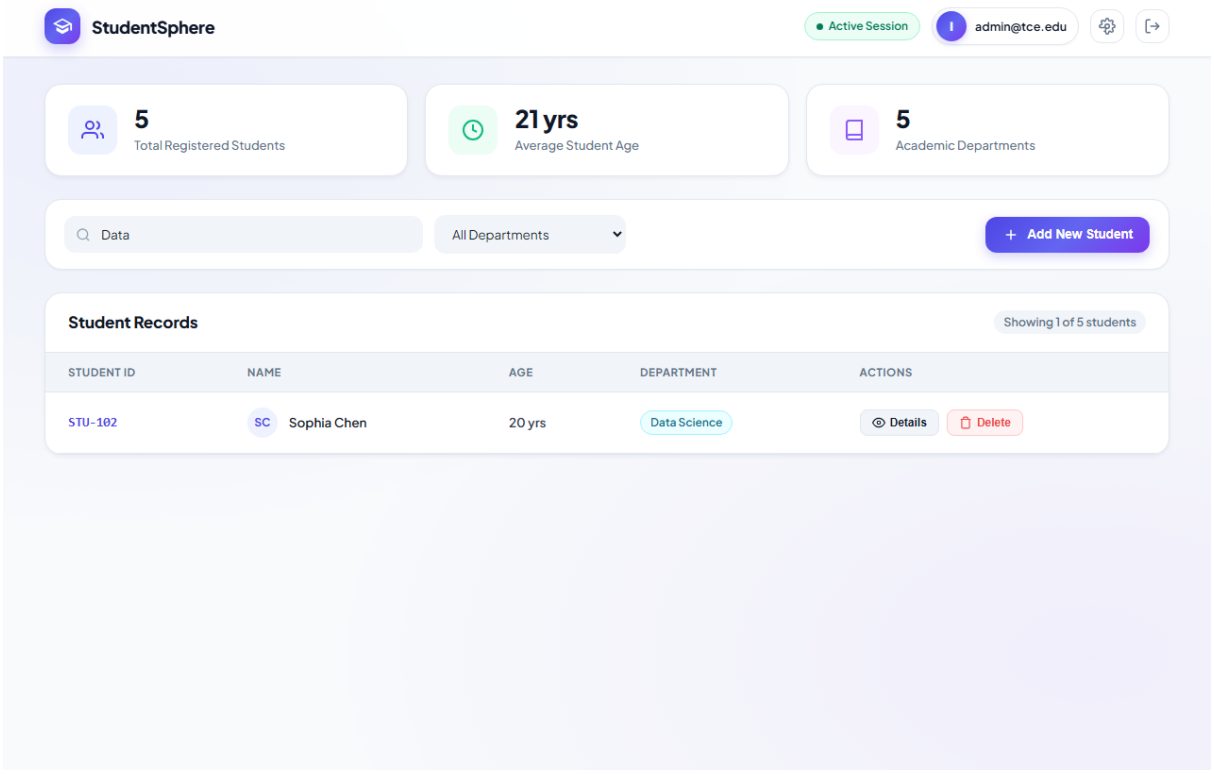


Figure 6: Search and department filtering in the dashboard.

10.5 Firebase Data Integration

The data module dynamically loads Firebase Firestore and Cloud Storage SDK modules. It reads student documents from the students collection and caches successful results locally. Student writes and deletes are synchronized to Firestore and Storage when available. Cloud Storage is used for the JSON backup path described in the project documentation.

10.6 Dropbox Integration

Dropbox settings store a user-provided access token and backup path in browser configuration. The service uses Dropbox Files API v2 operations for connection testing, downloading, uploading, and maintaining the students.json backup. A Dropbox token is a credential and should be entered only for a trusted account and rotated if it has been exposed.

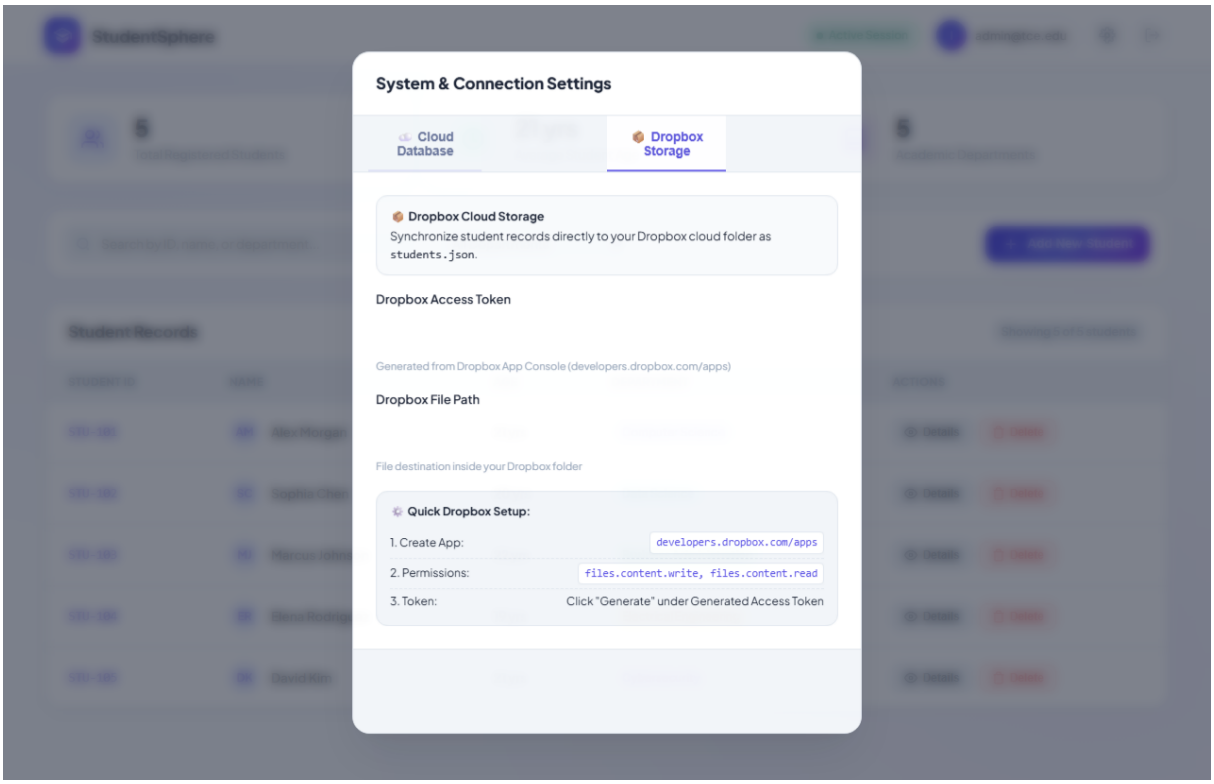


Figure 7: Dropbox provider settings.

10.7 MongoDB Atlas Integration

The MongoDB service supports connection testing, student retrieval, upsert, deletion, and bulk synchronization. In a hosted Firebase deployment, the mongodbApi Cloud Function validates a Firebase ID token before processing requests and loads the database URI from the MONGODB_URI Secret Manager secret. The database and collection defaults are studentsphere_db and students.

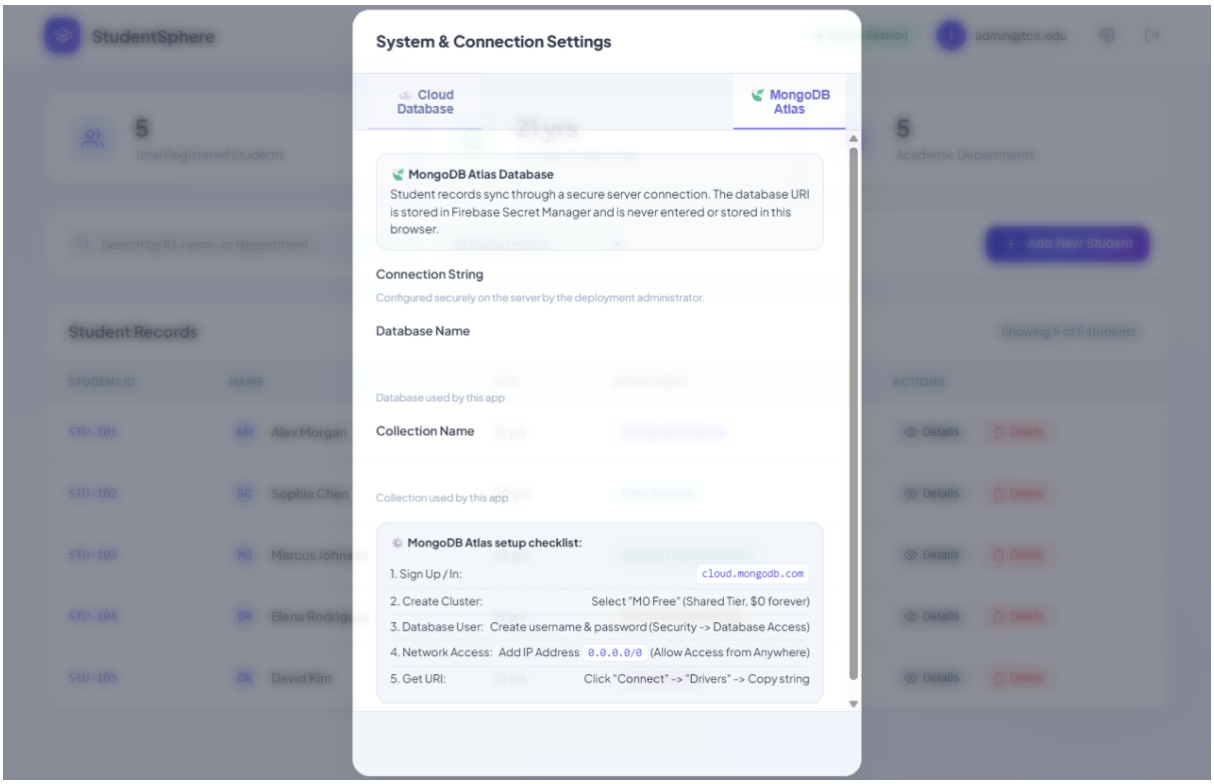


Figure 8: MongoDB Atlas connection and synchronization settings.

11. USER INTERFACE

11.1 Login and Registration

The authentication view introduces the StudentSphere brand and presents separate sign-in and registration forms. Email address and password fields use browser input types and required-field constraints. The registration flow additionally requests a full name and password confirmation.

11.2 Dashboard

The dashboard places summary metrics above a student table. The table includes the student ID, name, age, department, and action controls. An empty state and add-student action help first-time users proceed when there are no records.

11.3 Dialogs and Feedback

Modal dialogs separate add, view, delete, and provider configuration tasks from the main dashboard. Overlay clicks and close controls dismiss dialogs. Toast messages communicate status without navigating away from the current view.

12. TESTING AND VALIDATION

12.1 Acceptance Test Scenarios

The following scenarios describe a practical manual validation checklist. Cloud-specific scenarios require valid provider credentials, enabled services, and the appropriate security rules. No external provider result is assumed when credentials are not configured.

TEST	ACTION	EXPECTED RESULT
T-01	Open the application and load the sign-in view.	Authentication form and StudentSphere branding are displayed.

TEST	ACTION	EXPECTED RESULT
T-02	Register, sign in, and sign out with an enabled identity provider.	Authentication state changes and the dashboard is shown or hidden.
T-03	Add a student with all required fields.	The new record appears in the table and dashboard totals update.
T-04	Search by part of a name, ID, or department.	Only matching records remain visible and the result count changes.
T-05	Select a department and open a record's details.	The table is filtered and the selected record's fields are displayed.
T-06	Cancel and confirm a delete operation.	Cancel preserves the record; confirmation removes it from the view.
T-07	Configure a cloud provider and test its connection.	The UI reports the provider connection outcome.
T-08	Call the hosted MongoDB API without a Firebase ID token.	The function rejects the request with an authentication response.

12.2 Validation Considerations

- Verify record values in the UI and in each configured persistence target.
- Test offline or unavailable-provider behavior and confirm the local cache remains usable.
- Confirm Firebase Hosting rewrites MongoDB API requests to the correct function and region.
- Check that private MongoDB connection data is supplied through Secret Manager, not frontend files.

13. DEPLOYMENT

13.1 Local Development

The project README documents starting the Node.js server with npm start from the application directory. The local server serves the frontend at http://localhost:3000 and exposes development MongoDB routes. A local MONGODB_URI belongs in a private .env file and must not be committed.

13.2 Firebase Hosting and Cloud Function

firebase.json serves the frontend from the project directory and rewrites /api/mongodb/** to mongodbApi in us-central1. The function is configured for Node.js 22. Deployment requires Firebase CLI setup, the relevant Firebase services, and a billing-enabled plan for Cloud Functions. The MongoDB URI must be configured using Firebase Secret Manager before deploying the function.

13.3 Third-Party Provider Configuration

Firebase and Auth0 require valid application settings and allowed callback or authorized domains. Dropbox requires an app token with the selected file permissions. MongoDB Atlas requires database credentials and network access. Use least-privilege permissions and do not publish credentials in the frontend.

14. SECURITY AND PRIVACY

14.1 Authentication and API Access

Firebase Authentication or Auth0 establishes the user session. The hosted MongoDB Cloud Function extracts a bearer token and verifies it with Firebase Admin Auth before performing database operations. The MongoDB URI is read server-side from Secret Manager rather than delivered to the browser.

14.2 Credential Handling

- Keep database connection strings and provider access tokens private.
- Use Secret Manager or a private environment file for server-only credentials.
- Use restrictive Firebase Storage rules and avoid public read/write rules outside disposable testing.
- Rotate any credential that has been committed, shared, or exposed.
- Restrict Dropbox scopes and MongoDB database privileges to the application's needs.

14.3 Data Considerations

Student names and other identifying information should be treated as sensitive institutional data. The sample records in the project are illustrative. A production deployment should define data retention, authorization roles, audit logging, encryption requirements, and applicable privacy obligations.

15. LIMITATIONS

15.1 Current Limitations

- The record model contains only an ID, name, age, and department.
- The current interface is an administrative dashboard rather than a student self-service portal.
- Provider synchronization depends on account setup, network access, quotas, and valid credentials.
- Local demonstration authentication is not a replacement for production identity management.
- The application does not currently describe role-based authorization or a comprehensive audit trail.
- An institutional deployment needs stricter service rules, operational monitoring, and tested recovery procedures.

16. FUTURE ENHANCEMENTS

16.1 Planned Improvements

- Introduce user roles and authorization policies for administrators and faculty.
- Expand student records with course enrollment, contact information, and academic history.
- Add edit/update workflows, pagination, sorting, and export formats.
- Add audit logs, validation rules, and duplicate identifier prevention.
- Provide automated tests for UI flows, storage behavior, and protected API routes.
- Add sync conflict handling, retry visibility, and per-provider health indicators.
- Improve accessibility with keyboard focus management and screen-reader announcements.

17. CONCLUSION

StudentSphere implements a practical student management workflow with authentication, a searchable dashboard, student record operations, dashboard metrics, and configurable cloud persistence. Its modular structure demonstrates how a browser application can work with Firebase services and optional Auth0, Dropbox, and MongoDB Atlas integrations.

The project also illustrates an important boundary in cloud application design: browser-facing configuration must be distinguished from server credentials. The hosted MongoDB integration applies Firebase token verification and Secret Manager for the database URI. With stricter authorization, operational testing, and privacy controls, the current prototype can be extended toward a more complete institutional system.

18. REFERENCES

[1] Firebase Documentation, Authentication. https://firebase.google.com/docs/auth

[2] Firebase Documentation, Cloud Firestore. https://firebase.google.com/docs/firestore

[3] Firebase Documentation, Cloud Storage for Firebase. https://firebase.google.com/docs/storage

[4] Firebase Documentation, Hosting and Cloud Functions. https://firebase.google.com/docs/hosting and https://firebase.google.com/docs/functions

[5] Auth0 Documentation, Single Page Application SDK. https://auth0.com/docs/libraries/auth0-single-page-app-sdk

[6] Dropbox API Documentation, Files endpoints. https://www.dropbox.com/developers/documentation/http/documentation

[7] MongoDB Documentation, Node.js Driver and Atlas. https://www.mongodb.com/docs/drivers/node/current/ and https://www.mongodb.com/docs/atlas/

[8] Node.js Documentation, HTTP module. https://nodejs.org/api/http.html

[9] StudentSphere project source and README, Department of Information Technology project folder.

Online documentation accessed 19 October 2026.